ЛР 2: куда смотреть в коде

Шпаргалка к вопросу преподавателя «покажите, как у вас сделано X». Подробные объяснения и вопросы по теории лежат в lab2-defense.md, сценарий показа в DEMO.md. Здесь только быстрая навигация: что открыть, что сказать, где подвох.

Как открыть быстро

- Cmd+P: ввести имя файла (`ScriptSystem.cpp`).
- Cmd+Shift+O: список функций открытого файла. Cmd+T: поиск функции по всему проекту.
- Cmd+G: перейти на строку. Номера ниже сверены 9 октября 2026, но после любой правки они поплывут, поэтому надёжнее искать по имени функции.
- В каждой важной функции сверху есть комментарий с шагами. Прочитайте его вслух, это и есть ответ.

Три файла, которые нужно знать наизусть: `src/scripting/ScriptSystem.cpp`, `src/editor/EditorContext.cpp` (Play, Stop, hot reload) и `assets/scripts/waves.lua`.

1. Пять требований ЛР и где они в коде

Требование	Где открыть	Что сказать
Скриптовая подсистема	`ScriptSystem.h:66` класс, `ScriptSystem.cpp:356` `Impl`	Один встроенный Lua 5.4 (`sol::state`) на редактор. Lua собран из исходников прямо в exe (`cmake/LuaRuntime.cmake:36`), ничего ставить на машину не нужно
Двусторонние биндинги	Из C++ в Lua: `LuaBehaviour::call` `ScriptSystem.cpp:230`. Из Lua в C++: `bindEngine` `ScriptSystem.cpp:256`	C++ зовёт `on_create`, `on_update`, `on_destroy` у Lua-класса. Lua зовёт `get_position`, `spawn_prefab`, `destroy` и другие методы движка
Hot reload	`ScriptWatcher.cpp:50` `scan`, `EditorContext.cpp:201` опрос, `EditorContext.cpp:372` применение, `ScriptSystem.cpp:629` `hotReload`	Сохранил файл, вотчер на фоновом потоке заметил и проверил синтаксис, главный поток подменил класс. Игра не останавливается
Механика целиком на скриптах	`assets/scripts/waves.lua:32`, `enemy.lua:21`, `core.lua:21`	Волны, урон, cooldown и поражение считаются в Lua. C++ только даёт API и не знает правил игры
Шаблонный спавн (prefab)	`PrefabManager.cpp:51` `spawn`, `assets/prefabs/*.json`, `ScriptSystem.cpp:314` `spawn_prefab`	Префаб это JSON-описание сущности. Враг создаётся одной командой `world:spawn_prefab(путь, позиция)`

2. «Покажите, как у вас...»

Запуск и жизнь скрипта

> **...запускаются скрипты при нажатии Play.**
> `EditorContext::play` `EditorContext.cpp:1212`, затем `ScriptSystem::start` `ScriptSystem.cpp:702`, затем `Impl::create` `ScriptSystem.cpp:514`, внутри вызывается `on_create`. Сказать: перед запуском снимается снимок сцены, потом создаются экземпляры скриптов для всех сущностей со `ScriptComponent`. Если любой `on_create` упал, Play не начинается и сцена возвращается из снимка.

...каждый кадр вызывается on_update.

`EditorContext::updateGameplay` `EditorContext.cpp:1408`, там `scripts.update` `:1410`, затем `ScriptSystem::update` `ScriptSystem.cpp:729`, затем `LuaBehaviour::call` `:230`. Сказать: берётся список сущностей по возрастанию id, у каждой зовётся `on_update(dt)`. Скрипты идут первыми, потом ввод, физика, вращение, камера.

...C++ вызывает метод Lua.

`LuaBehaviour::call` `ScriptSystem.cpp:230`. Сказать: метод ищется в самом классе (`raw_get`), поэтому необъявленный колбэк просто ничего не делает. Вызов идёт через `protected_function` с обработчиком ошибок (стек), бюджетом времени (`Deadline`, строка 238) и зоной Tracy.

...Lua вызывает C++.

`bindEngine` `ScriptSystem.cpp:256`: типы `Vec3`, `Entity`, `World`. Сказать: любой метод скрипта, например `entity:get_position()`, это лямбда в этой функции. Каждый метод `Entity` сначала проверяет handle.

...аналог trampoline из лекции.

`ScriptBehaviour` (`ScriptSystem.h:50`, C++ база с виртуальными методами) и `LuaBehaviour` (`ScriptSystem.cpp:216`, адаптер). Сказать: это то же самое, что `PyBehaviourTrampoline` на слайде 58, только вместо `PYBIND11_OVERRIDE` свой вызов Lua. Отличие: колбэки у нас не чисто виртуальные, а пустые по умолчанию.

...Lua-класс наследует C++ базу.

`function uzlezz.behaviour` `ScriptSystem.cpp:331` (маленький Lua-кусок внутри `bindEngine`) и проверка `Impl::extract` `ScriptSystem.cpp:415`. Сказать: `uzlezz.behaviour {...}` запоминает базу в поле `base` и наследует методы через метатаблицу. `extract` потом проверяет, что класс объявлен именно так, что есть таблица `fields` и что колбэки это функции.

Ссылки на объекты

...что скрипт получает вместо сущности.

`EntityHandle` `ScriptSystem.cpp:28`, `World::generation` `World.h:29`, `World::clear` `World.cpp:55`. Сказать: не указатель, а id плюс номер поколения плюс слабая ссылка на «жизнь» мира. После удаления сущности, Stop или очистки мира `is_alive()` даёт false, остальные методы бросают «Stale EntityHandle» (строка 36). Это слайд 24 лекции: handle с уникальным id против устаревших ссылок. Тест: `ScriptSystemTests.cpp:100`.

...как удаляется враг.

Привязка `destroy` `ScriptSystem.cpp:322` и применение в `ScriptSystem::update` `:748`. Сказать: `world:destroy` только помечает сущность. Реально её удаляют после обхода всех скриптов, чтобы не сломать цикл и не убрать объект из-под соседнего скрипта в том же кадре. Помеченные не находятся поиском по тегу.

...как враг появляется из Lua.

Цепочка из четырёх шагов: `waves.lua:54` (`spawn_prefab`), `ScriptSystem.cpp:314`, `EditorContext.cpp:138` (сюда редактор подставил услугу), `PrefabManager::spawn` `PrefabManager.cpp:51`. Сказать: скриптовая подсистема сама ничего не знает про префабы, редактор подставляет ей функцию снаружи. Экземпляр скрипта у нового врага появляется в следующем кадре.

Hot reload

...как работает hot reload, по шагам.

1. `ScriptWatcher::poll` `ScriptWatcher.cpp:114` раз в 200 мс ставит фоновую задачу в job system.

2. `ScriptWatcher::scan` `ScriptWatcher.cpp:50` на воркере сравнивает время записи и размер файла, ждёт, пока отпечаток не изменится два опроса подряд, читает файл и проверяет синтаксис.
3. `EditorContext::update` `EditorContext.cpp:201` : главный поток забирает готовое через `drain` (`ScriptWatcher.cpp:134`) и вызывает `applyScriptChange` (`EditorContext.cpp:372`).
4. `ScriptSystem::hotReload` `ScriptSystem.cpp:629` заменяет классы и, если идёт Play, пересоздаёт экземпляры.

...при чём тут job system из ЛР 1.

`ScriptWatcher.cpp:125` (`submitBackground`), `JobSystem.cpp:181` . Сказать: обход папок, чтение файла и компиляция идут фоновой задачей, кадр не ждёт диск. Тест: `ScriptRuntimeTests.cpp:285` .

...чем отличаются L1 и L2.

`hotReload` `ScriptSystem.cpp:629` , `transfer` `ScriptSystem.cpp:471` , `on_reload` (строки 220 и 225). Сказать: у каждого экземпляра новый объект и новый `on_create` . L1 оставляет состояние из `on_create` . L2 переносит поля `self.*` из старого объекта, кроме тех, чьё начальное значение поменяли в коде (иначе правка `cooldown = 5` была бы не видна). Класс может взять перенос на себя через `on_reload(old)` . Тест: `ScriptRuntimeTests.cpp:211` . Важно не путать два места хранения. L1 и L2 касаются только `self.*` (например `self.cooldown` у ядра, `self.wave` и `self.remaining` у волн). Поля из `fields` (например `health` ядра) лежат в компоненте, `hotReload` их не трогает (только добавляет недостающие и выбрасывает удалённые), поэтому снятое HP остаётся и в L1, и в L2. Разница видна на волнах: в L2 номер волны и недоспавненные враги сохраняются, в L1 `Waves:on_create` обнуляет их и счёт идёт заново.

...что будет при синтаксической ошибке.

`ScriptWatcher::validate` `ScriptWatcher.cpp:24` , сообщение формирует `applyScriptChange` `EditorContext.cpp:372` . Сказать: файл только компилируется в отдельном временном `lua_State` , поэтому основная виртуальная машина не страдает. Ошибка с `файл:строка` уходит в Console и Gameplay, прежний код продолжает работать. Если ошибка не синтаксическая, а в структуре класса, её ловит `extract` до замены, результат тот же.

Ошибки и песочница

...что будет, если скрипт упал в рантайме.

`ScriptSystem::update` `ScriptSystem.cpp:729` , `failed = true` на строке 744, `traceback` `:129` , `locatedException` `:122` . Сказать: упавший экземпляр отключается (`failed`), остальные работают. В UI видна первая строка ошибки с `файл:строка` , в лог идёт полная со стеком. После исправления файла экземпляр оживает при hot reload.

...бесконечный цикл в скрипте.

`budgetHook` `ScriptSystem.cpp:186` , `lua_sethook` `:380` , `Deadline` `:197` . Сказать: раз в 1000 инструкций Lua вызывает хук, он сверяет время. Бюджет 100 мс на один вызов. Показать наглядно: раскомментировать `--while true do end` в `core.lua` . Тест: `ScriptRuntimeTests.cpp:133` .

...ограничение памяти.

`allocate` `ScriptSystem.cpp:168` , `Sandbox` `:161` . Сказать: Lua просит память только через наш распределитель, он считает объём и отказывает при превышении потолка 256 МБ. Тест: `ScriptRuntimeTests.cpp:173` .

...что скрипту запрещено.

`ScriptSystem.cpp:381` (`open_libraries`) и `:385` (`dofile` , `loadfile` обнулены). Сказать: открыты только base, math, table, string. Нет `io` , `os` , `debug` , `package` . Честная оговорка: это не полная изоляция, запись в `_G` переживает reload, игровое состояние в `_G` хранить нельзя.

Данные: префабы и поля

...как префаб становится сущностью.

`PrefabManager::spawn` `PrefabManager.cpp:51`, откат при ошибке `:101`. Сказать: JSON читается и проверяется заранее, затем создаётся сущность и по очереди добавляются компоненты. Если что-то бросило исключение, сущность целиком удаляется.

...откуда у скрипта поля и как их видит Inspector.

`ScriptComponent` `ScriptComponent.h:14`, `Impl::defaults` `ScriptSystem.cpp:490`, `set_field` `:284`, `InspectorPanel::drawScript` `InspectorPanel.cpp:728`. Сказать: таблица `fields` в Lua-классе это «схема типа»: имена, типы, значения по умолчанию (слайд 21 лекции). Значения из JSON ложатся поверх. Inspector рисует нужный виджет по типу значения. Тип поля менять нельзя. Поля лежат в компоненте, а не в `self`, поэтому их видят Inspector и сохранение в префаб.

...Save to Prefab.

`PrefabManager::saveFields` `PrefabManager.cpp:110`, `EditorContext::savePrefabFields` `EditorContext.cpp:409`. Сказать: запись идёт через временный файл и замену, поэтому прерванная запись не испортит префаб. Остальные ключи файла и их порядок сохраняются.

...как Play не ломает сцену.

`captureSnapshot` `EditorContext.cpp:1515`, `restoreSnapshot` `:1579`, `stop` `:1248`. Сказать: перед Play копируются компоненты всех сущностей, при Stop мир очищается и собирается заново с теми же id. Объекты Lua (`self`) в снимок не входят, поэтому его безопасно копировать.

Ввод и остальное

...как нажатие клавиши попадает в Lua.

`bindActions` `EditorContext.cpp:453`, `EditorContext.cpp:139`, `input_pressed` `ScriptSystem.cpp:293`, `InputManager::isActionPressed` `InputManager.cpp:159`, фокус окна `GameViewPanel.cpp:122`. Сказать: клавиши привязаны к именам действий (`StartWave` это Space, `DefensePulse` это F). Скрипт спрашивает действие по имени. Если окно Game не в фокусе, ввод для скриптов выключен.

...где логика волн, урона и кулдауна.

`waves.lua:32` (волны, статус, поражение), `enemy.lua:21` (движение и удар по ядру), `core.lua:21` (HP и импульс; поиск врагов в радиусе на строке 27). Сказать: в C++ нет ни одного решения про правила игры.

...как вы это проверяли.

`ctest` запускает 11 наборов. Скриптовые: `ScriptSystemTests` (игровой цикл на настоящих префабах: волны, урон, импульс, 100 циклов Play/Stop, устаревшие handle, ошибки, подмена кода, префаб) и `ScriptRuntimeTests` (список в `main` внизу файла: ошибки биндингов с местом вызова `:63`, приведение типов полей `:101`, бесконечный цикл `:133`, лимит памяти `:173`, hot reload `:211`, вотчер `:285`, stubs `:328`).

...подсказки в IDE.

`tools/lua-stubs/uzlezz.lua`, `.luarc.json`, тест `ScriptRuntimeTests.cpp:328`. Сказать: тест сверяет аннотации с реально привязанными методами и ловит и пропущенный, и лишний.

...Tracy.

Зона на каждый вход в скрипт: `ScriptSystem.cpp:234` (имя вида `Enemy:on_update`), графики `:755` (`Lua memory KB`, `Lua instances`).

3. Вопросы по лекции: где это у нас

Тема лекции	У нас	Где
Handle против устаревших указателей (слайд 24)	`EntityHandle`: id, поколение, жизнь мира	`ScriptSystem.cpp:28`
Spawner и type schema (слайд 21)	JSON-префаб и таблица `fields` в классе	`PrefabManager.cpp:51`, `ScriptSystem.cpp:490`
Скрипт как свойство `ScriptId` (слайд 16)	`ScriptComponent{path, className}`	`ScriptComponent.h:14`
Пакетное обновление (слайды 27-28)	Анимация делится на пакеты и раздаётся job system	`AnimationSystem.cpp:37`
Порядок подсистем в кадре (слайд 29)	Скрипты, ввод, физика, вращение, камера, потом анимация	`EditorContext.cpp:1408`
События (слайды 34-39)	`EventDispatcher`, одно событие `CollisionEvent`	`EventDispatcher.h:10`, `PhysicsSystem.cpp:259`
Запросы к объектам (слайд 25)	`find_by_tag`, `find_all_by_tag`, линейный перебор	`ScriptSystem.cpp:298`
Trampoline (слайд 58)	`LuaBehaviour`	`ScriptSystem.cpp:216`
Ошибки в пользовательском коде (слайд 63)	`protected_function`, стек, отключается один экземпляр	`ScriptSystem.cpp:230`, `:744`
Hot reload (слайд 64)	вотчер, проверка, транзакционная замена, L1 и L2	`ScriptWatcher.cpp:50`, `ScriptSystem.cpp:629`
Stubs (слайд 65)	`tools/lua-stubs/uzlezz.lua`	`ScriptRuntimeTests.cpp:328`
Дистрибуция (слайд 66)	Lua внутри exe	`cmake/LuaRuntime.cmake:36`
GIL (слайд 67)	У Lua его нет. Все вызовы Lua идут в главном потоке. Вотчер на воркере использует свой отдельный `lua_State`	`ScriptWatcher.cpp:24`

4. Чего нет, и как об этом честно сказать

Лучше сказать «этого нет, вот ближайшее, что есть», чем изображать, что есть.

- **События в Lua.** `EventDispatcher` есть, но скрипт не может на него подписаться. Столкновение доходит только до лога. Скрипты сейчас общаются опросом (враг каждый кадр ищет ядро). Это расширение: добавить в `bindEngine` подписку и пересылку `CollisionEvent`.
- **Быстрый поиск объектов.** `find_by_tag` перебирает все сущности, и `getEntities` (`World.cpp:69`) каждый раз копирует и сортирует список. На сотнях врагов это станет заметно. Лечится индексом по тегам.
- **Несколько скриптов на сущность и `AddComponent` из скрипта.** Сейчас один `ScriptComponent` на сущность. Скрипт может создать целую сущность через `spawn_prefab`, но не навесить компонент на чужую.
- **Корутины и `sleep`.** Библиотека `coroutine` не открыта. Вместо ожидания скрипты держат таймеры в полях (`self.timer` в `waves.lua`).

- **Параллельные скрипты.** Нет, и это сознательно: весь Lua и вся работа с миром идут в главном потоке, поэтому нет гонок. Параллельно работают только анимация, загрузка ресурсов и проверка файлов вотчером.
- **Полная изоляция скриптов.** Нет. Одна общая виртуальная машина, `_G` общий и переживает reload.
- **Отладчик с точками останова.** Нет. Есть `print` в лог с файлом и строкой, сообщения с `файл:строка` и зоны Tracy.
- **Граф целей, победа, главы.** Есть только волны и поражение в `waves.lua`.
- **Мировые чанки и стриминг мира.** Нет. Сцена грузится целиком, стримятся только ресурсы (меши, текстуры).

5. Если застрял на вопросе

1. Откройте `ScriptSystem.cpp` и прочитайте комментарий над ближайшей по смыслу функцией, он написан для этого.
2. Назовите путь вызова целиком, от кнопки до Lua: «кнопка, `EditorContext`, `ScriptSystem`, `LuaBehaviour::call`, Lua».
3. Если просят показать то, чего нет, смотрите раздел 4.